RUSTLAB BOLOGNA 2026 · 2 NOVEMBER

Burn, little candle, burn!

Machine learning in Rust with Candle and Burn

LUDWIG WEINZIERL



Two ends of one scale

CANDLE · CPU, 2 CORES

386 MB

SmolLM2-360M-Instruct, 8-bit

```
A tensor is a mathematical object
that generalizes the concept of a
vector, allowing for the extension
of vector calculus and linear
algebra to higher-dimensional
spaces.
```

BURN · NO_STD · RP2350 BOARD

1.1 MB

TinyStories 260K, f32, in 520 KiB of RAM

```
Once upon a time, there was a
little girl named Lily. She loved
to play outside in the park. One
day, she saw a big, red ball. She
wanted to play with it, but it was
too high.
```

Sizes of the weight files. Left: Candle 0.11 on a 2-core Linux VM. Right: Burn 0.22 without std on a Waveshare RP2350 board, 4.3 tokens/s; the same text in QEMU (mps2-an505) with RAM limited to 520 KiB.

Ludwig Weinzierl

- Rust since 2015
- Co-organizer, Rust Munich Meetup
- Embedded, LoRaWAN, retro computing
- Not an ML researcher
- Used Candle for MiniLM sentence embeddings in a comment score predictor

github.com/weinzierl

What does it take to do ML in Rust?

1. **Use someone else's model**
   an image, a sentence, a chat: Candle and the Hub
2. **Train one model in both**
   sin(x), from the same start
3. **Leave std behind**
   Burn on an emulated microcontroller, from sin(x) to a language model

Everything shown ran during preparation, on the platform each slide names.

Two frameworks

What does each one want to be?

01

What a framework does for you

Tensors

n-dimensional arrays, and fast kernels for them

Autodiff

gradients through tensor operations, for training

Weights in and out

files you can save, share and load

Backends

CPU, GPU, browser, microcontroller

Candle

"Minimalist ML framework for Rust with a focus on performance (including GPU support) and ease of use."

- PyTorch-like API: runtime shapes, errors as `Result`
- Goal: serverless inference, Python out of production
- `candle-transformers`: a model zoo, many also quantized (GGUF)
- CPU (MKL, Accelerate), CUDA, Metal, WebAssembly

Burn

"Burn is both a tensor library and a deep learning framework, optimized for numerical computing, training and inference."

- Training built in: optimizers, `Learner`, terminal dashboard
- CubeCL backends: CUDA, ROCm, Metal, Vulkan, WebGPU, CPU, with kernel fusion
- Flex: pure-Rust CPU backend, also for Wasm and `no_std`
- Imports ONNX (as generated Rust code), PyTorch and safetensors weights

Burn 0.22 moved the backend

UP TO 0.21

```
fn forward<B: Backend>(
    x: Tensor<B, 2>,
) -> Tensor<B, 2>

type B = Autodiff<NdArray>;
```

0.22

```
fn forward(
    x: Tensor<2>,
) -> Tensor<2>

let dev = Device::flex()
    .autodiff();
```

- The backend is a runtime `Device`, no longer a type parameter
- Burn's Candle backend removed; NdArray and LibTorch deprecated

Signatures only, to show the shape of the API. Many older tutorials show the old one. So does my abstract.

Tensors

Where does the information live?

02

One matrix product: Candle

```
let dev = Device::Cpu;
let m = [[1f32, 2.], [3., 4.]];
let a = Tensor::new(&m, &dev)?;
let b = a.matmul(&a)?;
println!("{b}");
```

OUTPUT

```
[[ 7., 10.],
 [15., 22.]]
Tensor[[2, 2], f32]
```

Operations that can fail return `Result<Tensor>`. The tensor knows its shape and element type at run time.

One matrix product: Burn

```
let dev = Device::flex();
let m = [[1., 2.], [3., 4.]];
let a = Tensor::<2>::from_floats(m, &dev);
let b = a.clone().matmul(a);
println!("{b}");
```

OUTPUT

```
Tensor {
  data:
[[7.0, 10.0],
 [15.0, 22.0]],
  shape:  [2, 2],
  device:  Device<Flex(Cpu)>,
  kind:  "Float",
  dtype:  "f32",
}
```

No `?`. The rank is in the type: `Tensor::<2>`. `matmul` takes `self`, so `a` is cloned to use it twice; the clone only bumps a reference count.

Wrong shapes: Candle returns an error

```rust
let a = Tensor::zeros((2, 3), DType::F32, &dev)?;
let b = Tensor::zeros((4, 5), DType::F32, &dev)?;
match a.matmul(&b) {
    Ok(c) => println!("{c}"),
    Err(e) => println!("Err: {e}"),
}
```

OUTPUT

```
Err: shape mismatch in matmul, lhs: [2, 3], rhs: [4, 5]
```

A shape error is a value. The program decides what happens next.

Wrong shapes: Burn panics

```
let a = Tensor::<2>::zeros([2, 3], &dev);
let b = Tensor::<2>::zeros([4, 5], &dev);
let c = a.matmul(b); // panics
println!("{c}");
```

OUTPUT (PANIC MESSAGE)

```
=== Tensor Operation Error ===
  Operation: 'Matmul'
  Reason:
    1. The inner dimension of matmul should be the same, but got 3 and 4
↳ . Lhs shape Shape { dims: [2, 3] }, rhs shape Shape { dims: [4, 5] }.
```

Burn treats it like a slice index out of bounds: a bug, so it panics. Check shapes from outside before the call.

Wrong rank: Burn does not compile

```rust
let a = Tensor::<2>::zeros([2, 3], &dev);
let v = Tensor::<1>::zeros([3], &dev);
let c = a.matmul(v);
```

$ cargo build

```
error[E0308]: mismatched types
   --> burn-demo/src/bin/burn_rank_error.rs:9:22
    |
  9 |     let c = a.matmul(v);
    |               ------ ^ expected `2`, found `1`
    |               |
    |               arguments to this method are incorrect
    |
    = note: expected struct `burn::Tensor<2>`
               found struct `burn::Tensor<1>`
```

`matmul` wants equal ranks: a `[3, 1]` matrix works. Candle catches this at run time.

Where the information lives

	Candle	Burn
rank	value	type: `Tensor<2>`
shape	value	value
element type	value: `DType::F32`	kind in the type (`Int`), width a value
device	value: `Device::Cpu`	value: `Device::flex()`
shape error	`Err(…)`	panic
operations	borrow: `&self`	move: `self`, cheap clone

Candle, batteries included

How far do a few lines get you?

03

A model is a download away

```rust
let weights = HFClientSync::new()?
    .model("timm", "resnet18.gluon_in1k")
    .download_file()
    .filename("model.safetensors")
    .send()?;
```

Program	Model, from the Hub	Size
`classify`	ResNet-18 · `timm/resnet18.gluon_in1k`	47 MB
`similar`	MiniLM · `sentence-transformers/all-MiniLM-L6-v2`	91 MB
`ask`	SmolLM2-360M-Instruct · `HuggingFaceTB`, 8-bit GGUF	386 MB

Downloaded once into `~/.cache/huggingface`, then read from there: with the network cut, all three programs still ran.

An image: ResNet-18

```rust
let bytes = fs::read(weights)?;
let vb = VarBuilder::from_buffered_safetensors(bytes, DType::F32, &device)?;
let model = resnet::resnet18(1000, vb)?;

let logits = model.forward(&load_image(&path, &device)?)?;
let probs = softmax_last_dim(&logits)?.squeeze(0)?.to_vec1::<f32>()?;
```



```
$ cargo classify 2-classify/images/cat.jpg
 81.7 %  Egyptian cat
   9.7 %  tiger cat
   8.3 %  tabby, tabby cat
   0.3 %  lynx, catamount
   0.0 %  washer, automatic washer, washing machine
```

The architecture comes from `candle-transformers`, the weights from the Hub. Scaling the image like the training images is your job.

A sentence: MiniLM

ONE VECTOR PER SENTENCE

```rust
let hidden = model.forward(&ids, &ids.zeros_like()?, Some(&mask))?;

// Average over the real tokens; padding must not count.
let mask = mask.to_dtype(DTYPE)?.unsqueeze(2)?;
let sum = hidden.broadcast_mul(&mask)?.sum(1)?;
let mean = sum.broadcast_div(&mask.sum(1)?)?;
Ok(mean.broadcast_div(&mean.sqr()?.sum_keepdim(1)?.sqrt()?)?)
```

COMPARE

```rust
// Unit vectors, so the dot product is the cosine similarity.
let scores = vectors.i(1..)?.matmul(&vectors.i(0)?.unsqueeze(1)?)?;
```

The model returns one vector per token. Averaging them into one vector per sentence, and making it length 1, is the part you write. Count the padding and it still compiles and runs: correct Rust, wrong embeddings.

Five sentences about Bologna

"WHERE CAN I EAT TONIGHT?"

```
 0.36  Tortellini are served till late.
 0.11  Fast trains run to Florence.
 0.09  Porticoes keep the streets dry.
 0.07  Two towers lean over the town.
 0.03  The university dates from 1088.
```

"IT IS RAINING."

```
 0.30  Porticoes keep the streets dry.
 0.09  Two towers lean over the town.
 0.02  Tortellini are served till late.
 0.02  Fast trains run to Florence.
-0.07  The university dates from 1088.
```

"HOW OLD IS THE CITY?"

```
 0.34  The university dates from 1088.
 0.25  Two towers lean over the town.
 0.23  Fast trains run to Florence.
 0.20  Porticoes keep the streets dry.
 0.17  Tortellini are served till late.
```

"HOW DO I RESET MY ROUTER?"

```
 0.05  The university dates from 1088.
 0.01  Porticoes keep the streets dry.
 0.00  Tortellini are served till late.
 0.00  Fast trains run to Florence.
-0.02  Two towers lean over the town.
```

Cosine similarity: 1 means the same direction. The tortellini sentence shares no word with its question. The router question still gets a top sentence: nearest is not relevant.

A chat: SmolLM2, one token at a time

```rust
for _ in 0..256 {
    let input = Tensor::new(&tokens[pos..], &device)?.unsqueeze(0)?;
    let logits = model.forward(&input, pos)?.squeeze(0)?;
    pos = tokens.len();
    let next = sampler.sample(&logits)?;
    if next == end {
        break;
    }
    tokens.push(next);
    if let Some(piece) = text.step(next).map_err(E::msg)? {
        print!("{piece}");
        io::stdout().flush()?;
    }
}
```

`pos` tells the model how much it has already seen: the KV cache inside `ModelWeights` keeps the rest. The prompt goes in once, then one token per step.

What it answers

THE CHAT TEMPLATE IS PLAIN TEXT

```
let prompt = format!(
    "<|im_start|>user\n{question}<|im_end|>\n<|im_start|>assistant\n"
);
```

$ cargo ask

```
A tensor is a mathematical object that generalizes the concept
of a vector, allowing for the extension of vector calculus and
linear algebra to higher-dimensional spaces.
```

31 tokens, 4.2 tokens/s on 2 CPU cores, 8-bit weights.

Sampling: top-p 0.9, temperature 0.5, seed 42. Same seed, same answer on this machine.
Metal and CUDA are cargo features.

What Candle gives you, what you write

Candle and friends	You
model code: ResNet, BERT, LLaMA	scale and normalize the image
weights from the Hub, cached	average the token vectors
tokenizers, the Rust library Python uses	the chat template
GGUF reader, sampler, KV cache	the loop that asks for the next token

`candle-transformers` 0.11.0 ships 100 model implementations and 22 quantized variants.

Learning sin(x)

Do they agree?

04

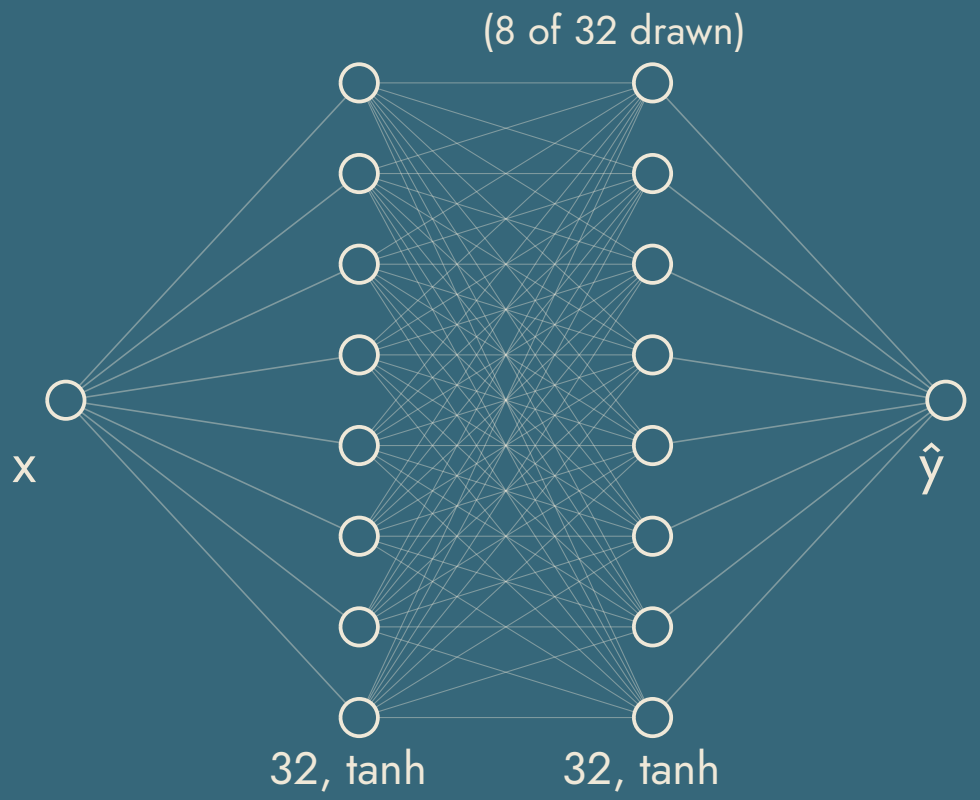
Learning, in four lines

predict	`ŷ = model(x, w)`	forward pass
measure	`loss = mean((ŷ − y)²)`	one number
blame	`g = ∂loss / ∂w`	autodiff
adjust	`w = w − lr · g`	optimizer

`w`: all weights, `lr`: learning rate. The simplest form: squared error and plain gradient descent. Every training run in this talk follows this loop, using Adam instead of plain descent; the classifiers use cross-entropy instead of squared error.

Experiment 1: learn sin(x)

- 256 points on [−π, π]
- Layers 1 → 32 → 32 → 1, tanh
- 1 153 parameters
- Adam, learning rate 0.01, 2 000 steps
- One shared start: `sine-init.safetensors`



The model, twice

CANDLE

```rust
fn forward(&self, x: &Tensor) -> Result<Tensor> {
    let x = self.l1.forward(x)?.tanh()?;
    let x = self.l2.forward(&x)?.tanh()?;
    self.l3.forward(&x)
}
```

BURN

```rust
fn forward(&self, x: Tensor<2>) -> Tensor<2> {
    let x = self.l1.forward(x).tanh();
    let x = self.l2.forward(x).tanh();
    self.l3.forward(x)
}
```

Same structure. Candle: borrow and `?`. Burn: move, no `Result`.

Training loop: Candle

```
let x = Tensor::from_vec(xs, (256, 1), &dev)?;
let y = x.sin()?;
let params = ParamsAdamW {
    lr: 0.01,
    weight_decay: 0.0,
    ..Default::default()
};
let mut opt = AdamW::new(varmap.all_vars(), params)?;
for step in 0..2000 {
    let loss = loss::mse(&model.forward(&x)?, &y)?;
    opt.backward_step(&loss)?;
    report(step, &loss)?;
}
```

`backward_step` runs the backward pass and the optimizer step. AdamW with weight decay 0 is plain Adam.

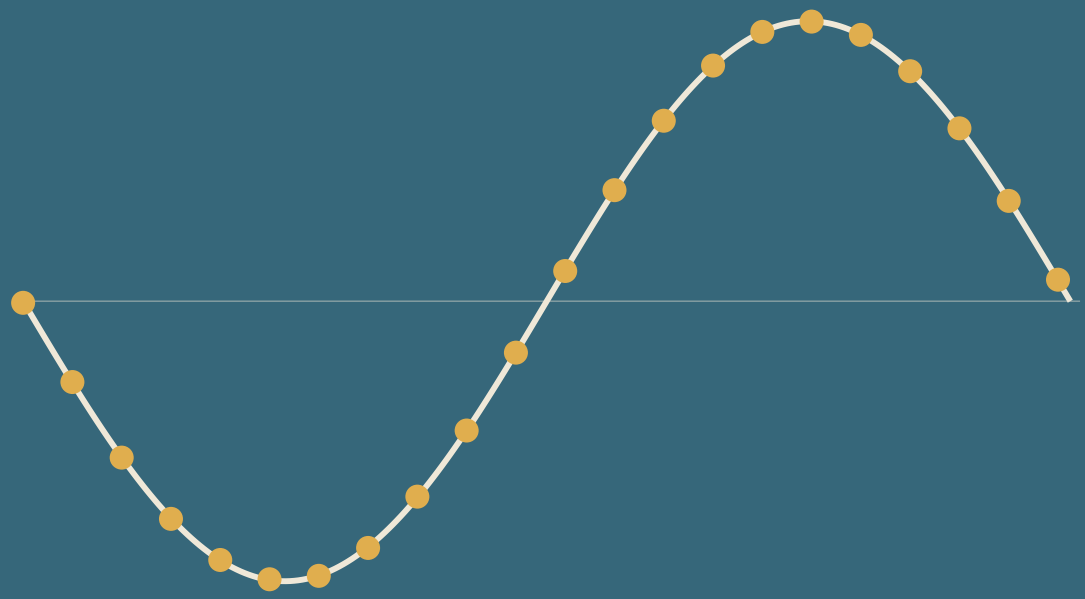
Training loop: Burn

```
let data = TensorData::new(xs, [256, 1]);
let x = Tensor::<2>::from_data(data, &dev);
let y = x.clone().sin();
let mut opt = AdamConfig::new().with_epsilon(1e-8).init();
for step in 0..2000 {
    let pred = model.forward(x.clone());
    let loss = MseLoss::new().forward(pred, y.clone(), Mean);
    let grads = loss.backward();
    let grads = GradientsParams::from_grads(grads, &model);
    model = opt.step(0.01, model, grads);
    report(step, loss);
}
```

The optimizer takes the model by value and returns the updated one. No `zero_grad`: gradients are consumed. Model and data live on `Device::flex().autodiff()`.

Same start, same data, same curve

step	Candle	Burn
0	6.588534	6.588533
250	0.001385	0.001385
500	0.000078	0.000078
750	0.000019	0.000019
1 000	0.000012	0.000012
1 250	0.000009	0.000009
1 500	0.000007	0.000007
1 750	0.000005	0.000005
2 000	0.000004	0.000004



line: sin(x) on [−π, π] · dots: Burn model · max error 0.0075 at the 256 training points

Loss (mean squared error) after every 250 updates; the 2 000 row is a check after the loop. The first loss differs in the last digit: floating point, likely Burn's SIMD adding in another order. Informal timing, five runs each on 2 vCPUs: Candle 526 to 685 µs per step, Burn 540 to 683 µs.

The detail that made it work

Candle, PyTorch

weight stored `[out, in]`
forward: `x · transpose(W) + b`

Burn

weight stored `[in, out]`
forward: `x · W + b`

BURN, LOADING THE SHARED START

```
let mut init = SafetensorsStore::from_file(INIT)
    .with_from_adapter(PyTorchToBurnAdapter);
model.load_from(&mut init).expect("load init");
```

Same `Linear` layer, transposed storage. `burn-store` adapters transpose on load (`PyTorchToBurnAdapter`) and on save (`BurnToPyTorchAdapter`).

Burn without std

How small can it go?

05



Four steps, one `#![no_std]` crate each

Each runs without std: on the desktop, as zero-import WebAssembly, and on an emulated Cortex-M33 limited to the Pico 2's 520 KiB of RAM, 256 KiB of it heap. Step 4 also ran on a real RP2350 board: same text, 4.3 tokens/s.

Candle is not in this part: it needs std. Its build for `thumbv8m.main-none-eabihf` stops at `getrandom`.

1
Inference

weights stay in flash

321 parameters

2
Training

autodiff on the device

1 316 parameters

3
Audio

sound, FFT, words

5 448 parameters

4
Pretrained LLM

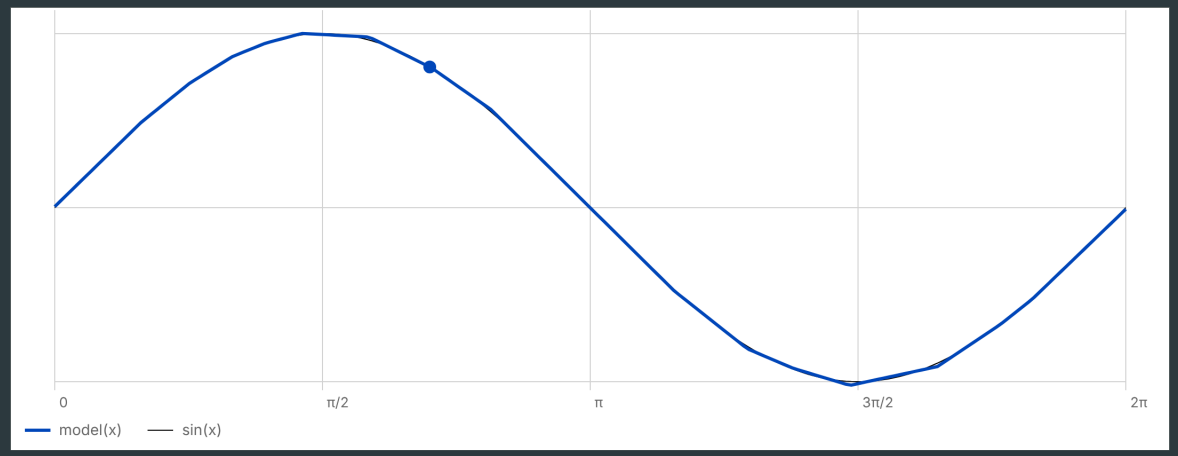
someone else's weights

TinyStories 260K

1 · Inference: the weights stay in flash

```
#[repr(C, align(256))]
struct Aligned<T: ?Sized>(T);
static ALIGNED: &Aligned<[u8]> =
    ↳ &Aligned(*include_bytes!("../weights/sine.bpk"));

/// The trained weights. On the microcontroller they stay in flash.
pub static WEIGHTS: &[u8] = &ALIGNED.0;
```



2 564
bytes of Burnpack, 1 284 of them weights

4 µs
per inference, desktop

0.0199
largest error, 25 points

7 888
bytes heap peak, emulated M33

2 · Training on the device

```
let mut learner = Learner::new(42);
for class in 0..4 {
    for _ in 0..5 {
        learner.add_example(&synth::window(class, &mut rng), class);
    }
}
let t = micros();
let loss = learner.train(15);
```

13 ms
15 epochs, 20 examples, desktop

100/100
new synthetic windows right

41 KiB
heap peak, training, emulated M33

`Learner` is the demo's own helper. Training and test windows come from one synthetic generator, as on the browser page. On the Pico 2 an IMU and a button would supply them: built, not yet run.

Two things std would have given

INSIDE TRAIN(): ONE EXAMPLE AT A TIME

```
let loss = loss_fn.forward(self.net.forward(x), y);
sum += loss.clone().into_scalar::<f32>();
self.net = self.adam.step(self.learning_rate, self.net.clone(),
    ↳ loss.backward());
```

- `burn-optim` 0.22.0 does not compile without std (6 errors, thumbv8m). Our `adam` is 78 lines: a `ModuleMapper` that updates each parameter.
- Without std on Arm, gemm assumes a 512 KiB L2 cache, and each small matrix-matrix product allocates a packing buffer that size. One example at a time keeps them matrix-vector products, which pack nothing.

Batch size 1 is the design, not laziness: a 512 KiB buffer cannot come out of a 256 KiB heap.

3 · Audio: teach it words

Audio
16 kHz, mic or file

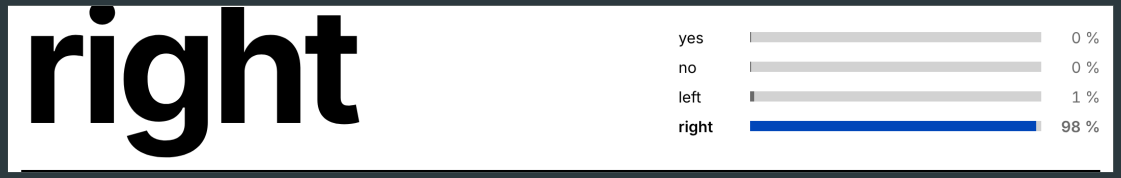
Log-mel
FFT, 16 bands

Words
loudness over the noise floor

Features
10 slices × 16 bands, plus length

MLP
5 448 parameters, trained here





167 ms
training, 20 clips, desktop

11/12
new synthetic words right

212 KiB
heap peak, emulated M33

Only the last box learns. The voice is synthetic (espeak-ng), not a person: real microphones are untested.

4 · Someone else's model, straight from flash

EVERY WEIGHT TENSOR BORROWS THE CHECKPOINT

```
let bytes = bytes::Bytes::from_static(&checkpoint[offset..offset + len]);
offset += len;
let bytes = Bytes::from_shared(bytes, AllocationProperty::Other);
Tensor::<2>::from_data(TensorData::from_bytes(bytes, [rows, cols],
    ↳ DType::F32), device)
```

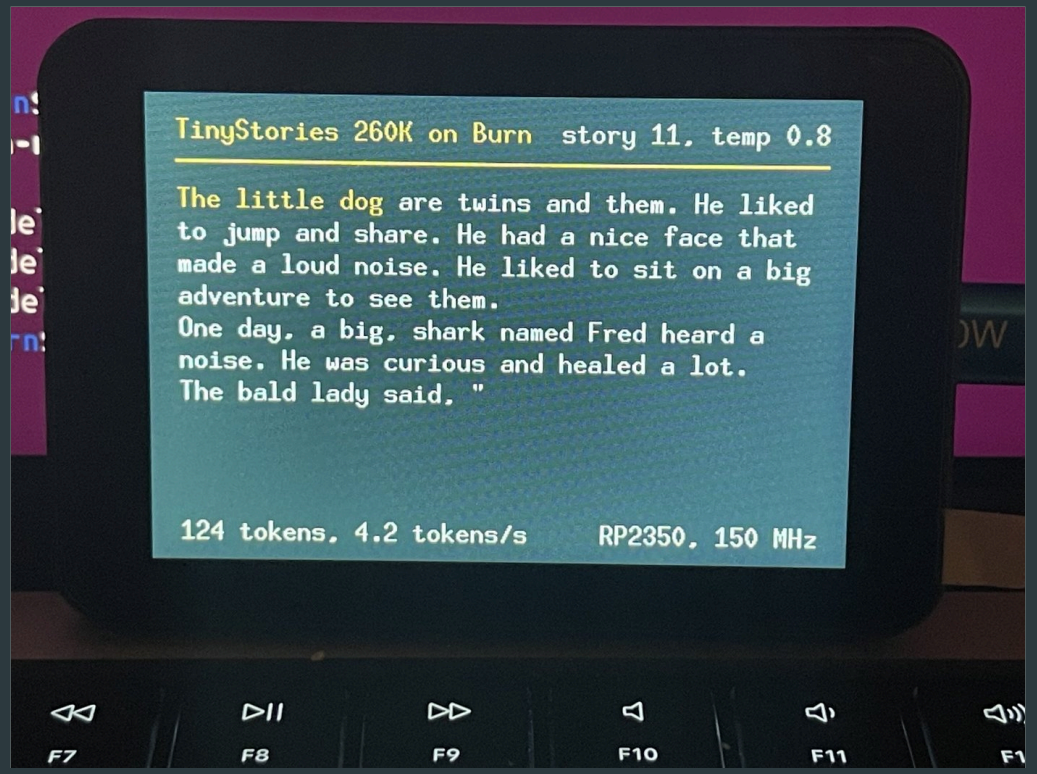
1.1 MB
of f32 weights, never copied

180 KiB
heap peak, 160 KiB of it KV cache

same text
as llama2.c's `run.c`

Karpathy's `stories260K.bin`, unchanged. Context cut to 128 of its 512 positions: at 512 the KV cache alone would take 640 KiB. At temperature 0 the text equals `run.c`'s for this prompt, 80 steps.

The same model on a real chip



4.3

tokens per second: 124 tokens in 28.5 s

same text

as in QEMU, all four stories

520 KiB

of RAM, no OS, no std

Photo of 1 November: story 11, temperature 0.8; the screen version takes the four prompts in turn. The numbers come from the USB log of `tinystories-usb`.

What it took

Problem	Fix
`web-time`, `wasm-bindgen-futures` pull in std	two small shims via `[patch.crates-io]`
`burn-optim` 0.22.0 does not compile without std	Adam in 78 lines
gemm's 512 KiB packing buffer per small product, no std, Arm	batch size 1, one token at a time
Burnpack wants 256-byte alignment	`#[repr(align(256))]` wrapper
no unwinder on a no_std Linux binary	two empty symbols

Burn's tensor, autodiff, nn, store and Flex crates build without std; burn-optim 0.22.0 does not. A whole no_std program needed these five fixes, all in user code.

Try it now, on your phone



weinzierl.github.io/burn-nostd

- **Gesture:** record each gesture five times, train, then move the phone
- **Keywords:** use the sample voice, train, play
- **TinyStories:** writes a story when it opens

The same Rust crates as the Pico 2 firmware. Nothing leaves the page.

Choosing

Which one, for what?

06

Which one?

	Reach for	Because
Run a model from the Hub	Candle	model zoo, tokenizers, GGUF
Train in Rust	Burn	`Learner` built in; Candle trains too
Train on the device	Burn	autodiff works without std
Microcontroller	Burn	Flex is `no_std`; Candle needs std
GPUs beyond CUDA, Metal	Burn	CubeCL: Vulkan, WebGPU
Import an ONNX model	Burn	`burn-onnx` writes Rust
Move weights between them	both	safetensors, same names; adapters

Three things I take home

1. **Candle: batteries included.** A model from the Hub is a download and a few lines. What you write is the part around it.
2. **Burn goes all the way down.** The same no_std crates train on a desktop and on an emulated Cortex-M33 in 520 KiB of RAM; a language model runs on a real RP2350.
3. **They agree.** Same start, same data: the printed losses match to six decimals from step 250 on.

Try it



github.com/weinzierl/
burn-little-candle-burn

Slides, notes, the Candle programs, both sin(x) runs

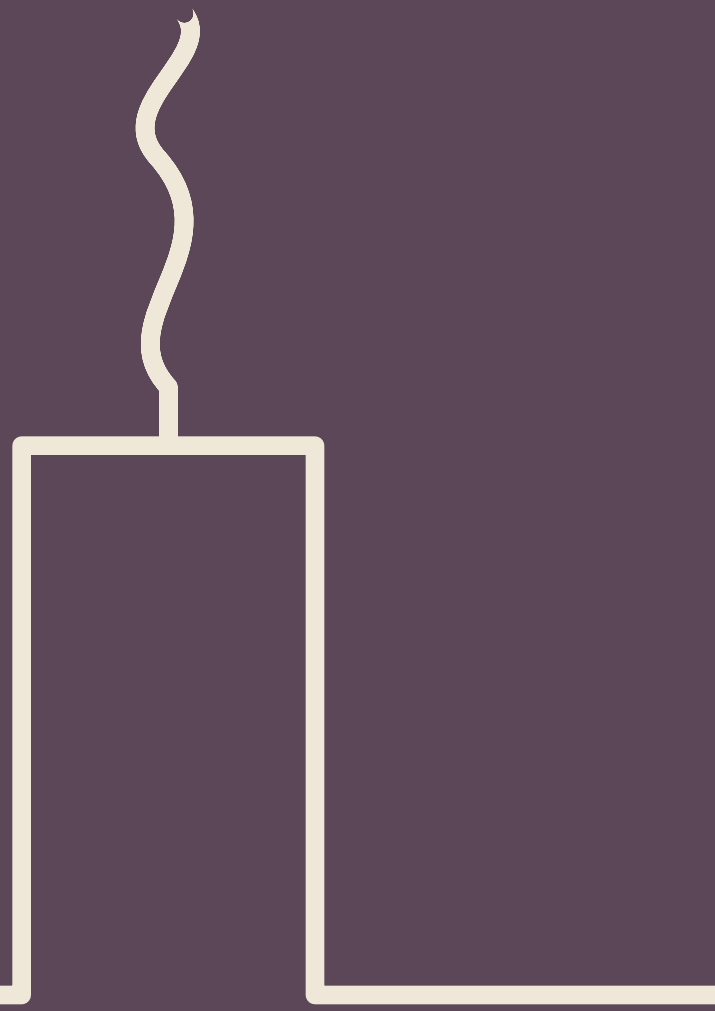


weinzierl.github.io/
burn-nostd

Burn without std: the four demos, code at github.com/weinzierl/burn-nostd

Grazie mille.

Questions?



SLIDES AND CODE

github.com/weinzierl/burn-little-candle-burn

Three ways a model comes in

WEIGHTS

Safetensors or GGUF: named tensors, no code. You bring the architecture: Candle's model zoo, or your own Burn module with burn-store, whose adapters convert PyTorch layouts.

GRAPH

ONNX. Burn: burn-onnx writes Rust source and a `.bpk` file at build time. Candle: candle-onnx walks the graph at run time. Check the operator lists first.

APPLICATION

Tokenizer, image scaling, pooling, chat template, sampling: your code, in both.

Loaded is not checked. Compare with a reference: TinyStories with llama2.c gives identical text, MiniLM with ONNX Runtime agrees within 0.0000003.

Quantization: a format and a kernel

CANDLE · GGUF Q8_0 · OUR SMOLLM2

- 32 weights share one f16 scale: 34 bytes, 8.5 bits a weight.
- 361.8 million weights: 384 MB of weights (the file: 386 MB), not 1.45 GB as f32.
- On the CPU the activations are quantized to the same blocks and multiplied as 8-bit integers.

BURN 0.22 · BURN BOOK

- Post-training quantization only, no quantization-aware training.
- Per tensor or per block, 8, 4 or 2 bits, on some backends.
- No integer operations yet: tensors are dequantized and computed in floating point.

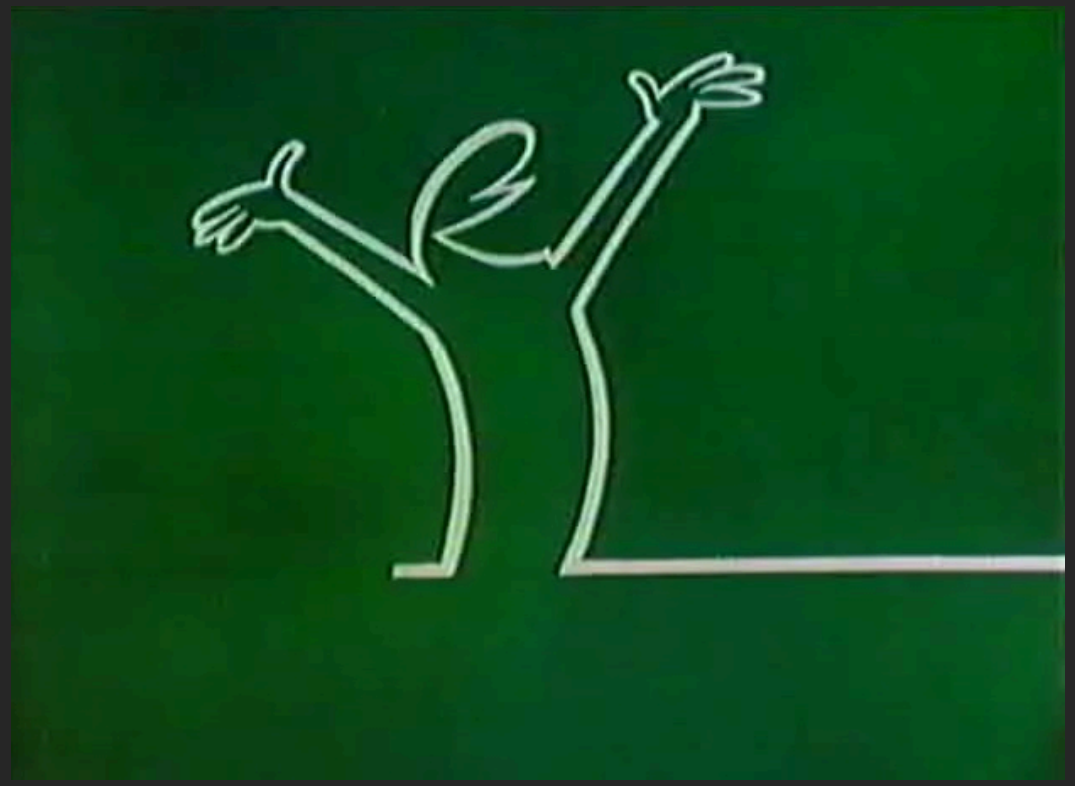
A smaller file is not automatically faster: the kernel decides.

Timing GPU code

- GPU work runs in the background: the call returns before the GPU is done.
- Warm up: Burn's GPU backends autotune by default, the first calls try several kernels and keep the fastest.
- Wait before you stop the clock: `device.synchronize()?` in Candle, `device.sync()?` in Burn. Reading a result back waits too.
- Burn's `device.profile(...)` measures device time; fusion holds work back, so end the closure with a read.
- Load once, time inference alone, repeat, take the median.

All times in this talk are CPU times from a shared VM: none is a benchmark.

Grazie, Cava



The design of these slides is a tribute to **Osvaldo Cavandoli** (1920 to 2007) and his **La Linea**.

La Linea © Osvaldo Cavandoli